

Text Preprocessing

NLP Toolkits and Preprocessing Techniques

- NLP Toolkits
 - Python libraries for natural language processing
- Text Preprocessing Techniques
 - Converting text to a meaningful format for analysis
 - Preprocessing and cleaning text

NLP Libraries

- NLTK (Natural Language Toolkit)
 - The most popular NLP library
- TextBlob
 - Wraps around NLTK and makes it easier to use
- spaCy
 - Built on Cython, so it's fast and powerful
- gensim
 - Great for topic modeling and document similarity

How to Install NLTK

Command Line

```
pip install nltk
```

Jupyter Notebook

```
import nltk
```

```
nltk.download()
```

```
# downloads all data & models
```

Sample Text Data

Hi Mr. Smith! I'm going to buy some vegetables (tomatoes and cucumbers) from the store. Should I pick up some black-eyed peas as well?

Text data is messy.

To analyze this data, we need to preprocess and normalize the text.

What is Text Preprocessing

- Text preprocessing is a method to clean the text data and make it ready to feed data to the model.
- Text data contains noise in various forms like emotions, punctuation, text in a different case.
- In other words, To preprocess your text simply means to bring your text into a form that is predictable and analyzable for your task.
- When we talk about Human Language then, there are different ways to say the same thing, and this is only the main problem we have to deal with because machines will not understand words, they need numbers so we need to convert text to numbers in an efficient manner.
- To analyze this data, we need to preprocess and normalize the text.

Preprocessing Techniques

1. Turn text into a meaningful format for analysis

- Tokenization

2. Clean the data

- Remove: capital letters, punctuation, numbers, stop words
- Stemming
- Parts of speech tagging
- Correct misspellings
- Chunking (named entity recognition, compound term extraction)

Techniques for Text Preprocessing

1) Expand Contractions

- Contraction is the shortened form of a word like don't stands for do not, aren't stands for are not. Like this, we need to expand this contraction in the text data for better analysis.
- you can easily get the dictionary of contractions on google or create your own and use the re module to map the contractions.
- Expand contraction function uses a regular expression to map the contraction to the word. It will match the word with keys and if it is present then replace that word with its value.

2) Lower Case

If the text is in the same case, it is easy for a machine to interpret the words because the lower case and upper case are treated differently by the machine.

for example, words like Ball and ball are treated differently by machine. So, we need to make the text in the same case and the most preferred case is a lower case to avoid such problems.

Another example where lowercasing is very useful is for search. Imagine, you are looking for documents containing “usa”. However, no results were showing up because “usa” was indexed as “USA”. Now, who should we blame?

Raw	Lowercased
Canada CanadA CANADA	canada
TOMCAT Tomcat toMcat	tomcat

'Hello everyone DONT BUY THIS PHONE at all first of all that says the phone is new i took it to the lab after month the phone is dead dead you can save it they open the phone in the lab and say s the phone is renew and its cheapest components i payed for only month now i need to buy new one this LG G is dead not a best thing people are saying to me dont buy from at all its troubling '



Lowecase

'hello everyone dont buy this phone at all first of all that says the phone is new i took it to the lab after month the phone is dead dead you can save it they open the phone in the lab and says the phone is renew and its cheapest components i payed for only month now i need to buy new one this lg g is dead not a best thing people are saying to me dont buy from at all its troubling '

3) Remove punctuations

One of the other text processing techniques is removing punctuations.

There are total 32 main punctuations that need to be taken care of. we can directly use the string module with a regular expression to replace any punctuation in text with an empty string.

32 punctuations which string module provide us is listed below.

'!"#\$%&'()*+,-./:;<=>?@[\\]^_`{|}~'

"Hello e-v-e-r-y-o-n-e!!!!@!!! ? ? @DONT BUY THIS PHONE at all-first o
f all that says the phone in new , i took it to the lab after 6 month the
phone is dead dead ,you can save it,they open the phone in the lab and say
s!!!! the phone is renew ,and its cheapest components.I payed 400\$ for on
ly 6 month ,now i need to buy new one this LG G4 is dead .not a best thing
'people are saying to me dont buy from at all, it's troubling!!!"



Removing irrelevant characters

'Hello e v e r y o n e DONT BUY THIS PHONE at all first o
f all that says the phone in new i took it to the lab after month the
phone is dead dead you can save it they open the phone in the lab and say
s the phone is renew and its cheapest components I payed for on
ly month now i need to buy new one this LG G is dead not a best thing
people are saying to me dont buy from at all it s troubling '

4) Remove words containing digits

- Sometimes it happens that words and digits combine are written in the text which creates a problem for machines to understand.
- Hence, We need to remove the words and digits which are combined like game57 or game5ts7.
- This type of word is difficult to process so better to remove them or replace them with an empty string.
- we use regular expressions for this.

5) Remove Stopwords

- Stopwords are the most commonly occurring words in a text which do not provide any valuable information. stopwords like they, there, this, where, etc are some of the stopwords.
- NLTK library is a common library that is used to remove stopwords and include approximately 180 stopwords which it removes.
- If we want to add any new word to a set of words then it is easy using the add method.

```
['hello', 'e', 'v', 'e', 'r', 'y', 'o', 'n', 'e', 'dont', 'buy', 'this',  
'phone', 'at', 'all', 'first', 'of', 'all', 'that', 'says', 'the',  
'phone', 'in', 'new', 'i', 'took', 'it', 'to', 'the', 'lab', 'after',  
'month', 'the', 'phone', 'is', 'dead', 'dead', 'you', 'can', 'save', 'it',  
'they', 'open', 'the', 'phone', 'in', 'the', 'lab', 'and', 'says', 'the',  
'phone', 'is', 'renew', 'and', 'its', 'cheapest', 'components', 'i',  
'payed', 'for', 'only', 'month', 'now', 'i', 'need', 'to', 'buy', 'new',  
'one', 'this', 'lg', 'g', 'is', 'dead', 'not', 'a', 'best', 'thing',  
'people', 'are', 'saying', 'to', 'me', 'dont', 'buy', 'from', 'at', 'all',  
'it', 's', 'troubling']
```



Removing Stopwords

```
['hello', 'e', 'v', 'e', 'r', 'n', 'e', 'dont', 'buy', 'phone', 'first',  
'says', 'phone', 'new', 'took', 'lab', 'month', 'phone', 'dead', 'dead',  
'save', 'open', 'phone', 'lab', 'says', 'phone', 'renew', 'cheapest',  
'components', 'payed', 'month', 'need', 'buy', 'new', 'one', 'lg', 'g',  
'dead', 'best', 'thing', 'people', 'saying', 'dont', 'buy', 'troubling']
```


6) Rephrase text

- we may need to modify some text or change the pattern to a particular string which makes it easy to identify like we can match the pattern of email ids and change it to string like email address.

7) Stemming and Lemmatization

- Stemming is a process to reduce the word to its root stem for example run, running, runs, runed derived from the same word as run.
- basically stemming do is remove the prefix or suffix from word like ing, s, es, etc. NLTK library is used to stem the words.
- The stemming technique is not used for production purposes because it is not so efficient technique and most of the time it stems the unwanted words. So, to solve the problem another technique came into the market as Lemmatization.
- there are various types of stemming algorithms like porter stemmer, snowball stemmer. Porter stemmer is widely used present in the NLTK library.

Lemmatization

- Lemmatization is similar to stemming, used to stem the words into root word but differs in working.
- Actually, Lemmatization is a systematic way to reduce the words into their lemma by matching them with a language dictionary.

I **saw** an amazing thing $\xrightarrow{\text{stem}}$ I **s** an amazing thing

I **saw** an amazing thing $\xrightarrow{\text{lemma}}$ I **see** an amazing thing

['hello', 'e', 'v', 'e', 'r', 'n', 'e', 'dont', 'buy', 'phone', 'first',
'says', 'phone', 'new', 'took', 'lab', 'month', 'phone', 'dead', 'dead',
'save', 'open', 'phone', 'lab', 'says', 'phone', 'renew', 'cheapest',
'commonents', 'payed', 'month', 'need', 'buy', 'new', 'one', 'lg', 'g',
'dead', 'best', 'thing', 'people', 'saying', 'dont', 'buy', 'troubling']

↓ Stemming

['hello', 'e', 'v', 'e', 'r', 'n', 'e', 'dont', 'buy', 'phone', 'first',
'say', 'phone', 'new', 'took', 'lab', 'month', 'phone', 'dead', 'dead',
'save', 'open', 'phone', 'lab', 'say', 'phone', 'renew', 'cheapest',
'common', 'pay', 'month', 'need', 'buy', 'new', 'one', 'lg', 'g',
'dead', 'best', 'thing', 'peopl', 'say', 'dont', 'buy', 'troubl']

['hello', 'e', 'v', 'e', 'r', 'n', 'e', 'dont', 'buy', 'phone', 'first',
'says', 'phone', 'new', 'took', 'lab', 'month', 'phone', 'dead', 'dead',
'save', 'open', 'phone', 'lab', 'says', 'phone', 'renew', 'cheapest',
'components', 'payed', 'month', 'need', 'buy', 'new', 'one', 'lg', 'g',
'dead', 'best', 'thing', 'people', 'saying', 'dont', 'buy', 'troubling']

↓
Lemmatization

['hello', 'e', 'v', 'e', 'r', 'n', 'e', 'dont', 'buy', 'phone', 'first',
'say', 'phone', 'new', 'take', 'lab', 'month', 'phone', 'dead', 'dead',
'save', 'open', 'phone', 'lab', 'say', 'phone', 'renew', 'cheapest',
'components', 'pay', 'month', 'need', 'buy', 'new', 'one', 'lg', 'g',
'dead', 'best', 'thing', 'people', 'say', 'dont', 'buy', 'trouble']

original_word stemmed_words

0	connect	connect
1	connected	connect
2	connection	connect
3	connections	connect
4	connects	connect

original_word stemmed_word

0	trouble	troubl
1	troubled	troubl
2	troubles	troubl
3	troublesome	troublesom

	original_word	lemmatized_word
0	trouble	trouble
1	troubling	trouble
2	troubled	trouble
3	troubles	trouble

	original_word	lemmatized_word
0	goose	goose
1	geese	goose

8) Remove Extra Spaces (Noise Removal)

- Most of the time text data contain extra spaces or while performing the above preprocessing techniques more than one space is left between the text so we need to control this problem. regular expression library performs well to solve this problem.

	raw_word	cleaned_word
0	..trouble..	trouble
1	trouble<	trouble
2	trouble!	trouble
3	<a>trouble	trouble
4	1.trouble	trouble

9) Remove the words having length ≤ 2 (Noise Removal)

```
['hello', 'e', 'v', 'e', 'r', 'n', 'e', 'dont', 'buy', 'phone', 'first',  
'say', 'phone', 'new', 'take', 'lab', 'month', 'phone', 'dead', 'dead',  
'save', 'open', 'phone', 'lab', 'say', 'phone', 'renew', 'cheapest',  
'components', 'pay', 'month', 'need', 'buy', 'new', 'one', 'lg', 'g',  
'dead', 'best', 'thing', 'people', 'say', 'dont', 'buy', 'trouble']
```



Removing word having less length

```
['hello', 'dont', 'buy', 'phone', 'first', 'say', 'phone', 'new', 'take',  
'lab', 'month', 'phone', 'dead', 'dead', 'save', 'open', 'phone', 'lab',  
'say', 'phone', 'renew', 'cheapest', 'components', 'pay', 'month', 'need',  
'buy', 'new', 'one', 'dead', 'best', 'thing', 'people', 'say', 'dont',  
'buy', 'trouble']
```

10) Normalization

Text normalization is the process of transforming a text into a canonical (standard) form.

For example, the word “goooooood” and “gud” can be transformed to “good”, its canonical form. Another example is mapping of near identical words such as “stopwords”, “stop-words” and “stop words” to just “stopwords”.

Text normalization is important for noisy texts such as social media comments, text messages and comments to blog posts where abbreviations, misspellings and use of out-of-vocabulary words (oov) are prevalent.

Raw	Normalized
2moro 2mrrw 2morrow 2mrw tomrw	tomorrow
b4	before
otw	on the way
:) :-) ;-)	smile

11) Text Enrichment / Augmentation

Text enrichment involves augmenting your original text data with information that you did not previously have. Text enrichment provides more semantics to your original text, thereby improving its predictive power and the depth of analysis you can perform on your data.

In an information retrieval example, expanding a user's query to improve the matching of keywords is a form of augmentation, it can help fetch documents that are more relevant.

You can use part-of-speech tagging to get more granular information about the words in your text. For example, in a document classification problem, the appearance of the word **book as a noun** could result in a different classification than **book as a verb** as one is used in the context of reading and the other is used in the context of reserving something.

12) Tokenization:

In this step, the text is split into smaller units. We can use either sentence tokenization or word tokenization based on our problem statement.

msg_lower	msg_tokenied
go until jurong point crazy available only in bugis n great world la e buffet cine there got amore wat	[go, until, jurong, point, crazy, available, only, in, bugis, n, great, world, la, e, buffet, cine, there, got, amore, wat]
ok lar joking wif u oni	[ok, lar, joking, wif, u, oni]
free entry in 2 a wkly comp to win fa cup final tkts 21st may 2005 text fa to 87121 to receive entry questionstd txt ratetcs apply 08452810075over18s	[free, entry, in, 2, a, wkly, comp, to, win, fa, cup, final, tkts, 21st, may, 2005, text, fa, to, 87121, to, receive, entry, questionstd, txt, ratetcs, apply, 08452810075over18s]
u dun say so early hor u c already then say	[u, dun, say, so, early, hor, u, c, already, then, say]
nah i dont think he goes to usf he lives around here though	[nah, i, dont, think, he, goes, to, usf, he, lives, around, here, though]

13) URL removal

URLs (or Uniform Resource Locators) in a text are references to a location on the web, but do not provide any additional information. We thus, remove these too using the library named `re`, which provides regular expression matching operations.

We take our sample text and analyse each word, removing words or strings starting with `http`.

Tokenization

Tokenization = splitting raw text into small, indivisible units for processing

These units can be:

- Words
- Sentences
- N-grams
- Other characters defined by regular expressions

Code: Tokenization (Words)

Input:

```
from nltk.tokenize import word_tokenize

my_text = "Hi Mr. Smith! I'm going to buy some vegetables (tomatoes and cucumbers)
from the store. Should I pick up some black-eyed peas as well?"

print(word_tokenize(my_text)) # print function requires Python 3
```

Output:

Tokenization: Sentences

Hi Mr. Smith! I'm going to buy some vegetables (tomatoes and cucumbers) from the store. Should I pick up some black-eyed peas as well?

Tokens can be sentences. How would you split this into sentences? What rules would you put in place?

It's a difficult task. This is where tokenizers in Python can help.

Code: Tokenization (Sentences)

Input:

```
from nltk.tokenize import sent_tokenize
```

```
my_text = "Hi Mr. Smith! I'm going to buy some vegetables (tomatoes and cucumbers)  
from the store. Should I pick up some black-eyed peas as well?"
```

```
print(sent_tokenize(my_text))
```

Output:

Tokenization: Regular Expressions

Let's say you want to tokenize by some other type of grouping or pattern.

Regular expressions (regex) allows you to do so.

Some examples of regular expressions:

- Find white spaces: `\s+`
- Find words starting with capital letters: `[A-Z]['\w]+`

Code: Tokenization (Regular Expressions)

```
from nltk.tokenize import RegexpTokenizer
```

```
my_text = "Hi Mr. Smith! I'm going to buy some vegetables (tomatoes" \
" and cucumbers) from the store. Should I pick up some black-eyed " \
"peas as well?"
```

```
# Regexp Tokenizer with whitespace delimiter
```

```
whitespace_tokenizer = RegexpTokenizer("\s+", gaps=True)
```

```
# my_text.decode('utf-8') <--- for python2 only
```

```
print(whitespace_tokenizer.tokenize(my_text))
```

```
['Hi', 'Mr.', 'Smith!', 'I'm', 'going', 'to', 'buy', 'some', 'vegetables',  
'(tomatoes', 'and', 'cucumbers)', 'from', 'the', 'store.', 'Should', 'I', 'pi  
ck', 'up', 'some', 'black-eyed', 'peas', 'as', 'well?']
```

Code: Tokenization (Regular Expressions)

Input:

```
from nltk.tokenize import RegexpTokenizer  
  
# RegexpTokenizer to match only capitalized words cap_tokenizer  
= RegexpTokenizer("[A-Z]['\w]+")  
print(cap_tokenizer.tokenize(my_text))
```

Output:

Tokenization Summary

Hi Mr. Smith! I'm going to buy some vegetables (tomatoes and cucumbers) from the store. Should I pick up some black-eyed peas as well?

With tokenization, we were able to break this messy text data down into small units for us to do analysis

- By sentence, word, n-grams
- By characters and patterns using regular expressions

Preprocessing Checkpoint

What have we done so far?

- Tokenized text by sentence, word, n-grams and using regex

This is only one step. There is a lot more preprocessing that we can do.

Preprocessing: Remove Characters

Hi Mr. Smith! I'm going to buy some vegetables (tomatoes and cucumbers) from the store. Should I pick up 2lbs of black-eyed peas as well?

How can we normalize this text?

- Remove punctuation
- Remove capital letters and make all letters lowercase
- Remove numbers

Code: Remove Punctuation

Input:

```
import re # Regular expression library
import string

# Replace punctuations with a white space
clean_text = re.sub('[%s]' % re.escape(string.punctuation), ' ', my_text) clean_text
```

Code: Make All Text Lowercase

Input:

```
clean_text = clean_text.lower()  
clean_text
```

Output:

Code: Remove Numbers

Input:

```
# Removes all words containing digits
```

```
clean_text = re.sub('\w*\d\w*', '', clean_text) clean_text
```

Output:

```
'hi mr      smith      i m going to buy some vegetables      tomatoes and cucumbers      from  
the store  should i pick up      of black eyed peas as well'
```

Regular Expressions

Basic Regular Expression Meta-Characters

Operator	Behavior
.	Wildcard, matches any character
^abc	Matches some pattern <i>abc</i> at the start of a string
abc\$	Matches some pattern <i>abc</i> at the end of a string
[abc]	Matches one of a set of characters
[A-Z0-9]	Matches one of a range of characters
ed ing s	Matches one of the specified strings (disjunction)
*	Zero or more of previous item, e.g. <i>a*</i> , <i>[a-z]*</i> (also known as <i>Kleene Closure</i>)
+	One or more of previous item, e.g. <i>a+</i> , <i>[a-z] +</i>
?	Zero or one of the previous item (i.e. optional), e.g. <i>a?</i> , <i>[a-z] ?</i>
{n}	Exactly <i>n</i> repeats where <i>n</i> is a non-negative integer
{n, }	At least <i>n</i> repeats
{, n}	No more than <i>n</i> repeats
{m, n}	At least <i>m</i> and no more than <i>n</i> repeats
a (b c) +	Parentheses that indicate the scope of the operators

Tips and Tricks: Lambda

```
# Basic example of a lambda
```

```
square_me = lambda x : x * x
```

```
print(square_me(5))
```

25

INPUT

OUTPUT

```
# Yes it is exactly equivalent to
```

```
def square_me_too(x):
```

```
    return x * x
```

```
print(square_me_too(5))
```

25

Tips and Tricks: Lambdas and Maps

```
# Basic example of a map
```

```
my_numbers = [9, 3, 4, 100, 2, 1]
```

```
my_numbers_squared = list(map(square_me, my_numbers))
```

```
print(my_numbers_squared)
```

```
[81, 9, 16, 10000, 4, 1]
```

Tips and Tricks: Lambdas and Maps

```
# But what if you want to clean a bunch of texts?
text1 = "I'm going to buy 5 cans of beans"
text2 = "I'm going to buy 6lbs of ham"
text3 = "I'm going to 111 Dore Street"

texts = [text1, text2, text3]

# I could use a for loop... or I can use a lambda and a map
remove_numbers = lambda x : re.sub('\w*\d\w*', ' ', x)

texts = list(map(remove_numbers, texts))

print(texts)
```

```
["I'm going to buy  cans of beans", "I'm going to buy  of ham", "I'm going  
to  Dore Street"]
```


Preprocessing: Stop Words

Hi Mr. Smith! I'm going to buy some vegetables (tomatoes and cucumbers) from the store. Should I pick up some black-eyed peas as well?

What is the most frequent term in the text above? Is that information meaningful?

Stop words are words that have very little semantic value.

There are language and context-specific stop word lists online that you can use.

Code: Stop Words

Input:

```
from nltk.corpus import stopwords  
set(stopwords.words('english'))
```

Output:

```
{'but', 'isn', 'under', 'weren', 'those', 'when', 'why', 'few', 'for', 'it', 'of', 'down', 'ma',  
'over', 'd', 'during', 'shouldn', 'did', 'above', 'below', 'myself', 'further', 'very', 'same',  
'too', 'does', 'through', 'from', 'didn', 'whom', 'and', 'am', 'such', 'out', 'or', 'me', 'has',  
'will', 'shan', 'on', 'then', 'here', 't', 'with', 'some', 'what', 'don', 'were', 'an',  
'themselves', 'yourselves', 'off', 'being', 'more', 'they', 'ourselves', 'into', 'my', 'them',  
'ain', 'a', 'wouldn', 'itself', 'i', 'hasn', 'her', 'their', 'mustn', 'our', 'herself', 'where',  
'hers', 'once', 'any', 'theirs', 'before', 'most', 'other', 'not', 'himself', 'his', 'if', 'he',  
'each', 'are', 'how', 'couldn', 'ours', 'doing', 'hadn', 'needn', 'again', 'these', 'wasn', 'nor',  
'do', 'just', 'so', 'we', 'there', 'have', 'by', 'o', 'than', 're', 'while', 'your', 'at', 'him',  
'own', 'can', 'you', 'll', 'between', 'been', 'that', 'is', 'she', 'yours', 'this', 'was', 'be',  
'had', 'doesn', 'no', 'because', 'won', 'both', 'to', 'against', 'aren', 'y', 'after', 'all', 'up',  
've', 'should', 'as', 'in', 'the', 'having', 'until', 'who', 'haven', 'only', 'm', 'yourself', 'about', 's', 'which', 'now', 'mightn', 'its'}
```

Code: Remove Stop Words

Input:

```
From sklearn. feature_extraction.text import CountVectorizer
my_text = ["Hi Mr. Smith! I'm going to buy some vegetables (tomatoes and cucumbers)
from the store. Should I pick up some black-eyed peas as well?"]

# Incorporate stop words when creating the count vectorizer
cv = CountVectorizer(stop_words='english') X =
cv.fit_transform(my_text)
pd.DataFrame(X.toarray(), columns=cv.get_feature_names())
```

Normalization

Need to “normalize” terms

Information Retrieval: indexed text & query terms must have same form.

We want to match **U.S.A.** and **USA**

We implicitly define equivalence classes of terms

e.g., deleting periods in a term

Alternative: asymmetric expansion:

Enter: **window** Search: **window, windows**

Enter: **windows** Search: **Windows, windows, window**

Enter: **Windows** Search: **Windows**

Potentially more powerful, but less efficient

Morphology

Morphology is the study of the internal structure of words, of the way words are built up from smaller meaning units.

Morpheme:

The smallest meaningful unit in the grammar of a language.

Two classes of morphemes

Stems: “main” morpheme of the word, supplying the main meaning (i.e. *establish* in the example below)

Affixes: add additional meaning

Prefixes: **Antidis**establishmentarianism

Suffixes: Antidisestablish**mentarianism**

Morphology

Morphemes:

The small meaningful units that make up words

Stems: The core meaning-bearing units

Affixes: Bits and pieces that adhere to stems

Often with grammatical functions

Morphology: examples

Unladylike

The word *unladylike* consists of three morphemes and four syllables.

Morpheme breaks:

un- 'not'

lady '(well behaved) female adult human'

-like 'having the characteristics of'

None of these morphemes can be broken up any more without losing all sense of meaning. *Lady* cannot be broken up into "la" and "dy," even though "la" and "dy" are separate syllables. Note that each syllable has no meaning on its own.

Dogs

The word *dogs* consists of two morphemes and one syllable:

dog, and

-s, a plural marker on nouns

Technique

The word *technique* consists of only one morpheme having two syllables.

Even though the word has two syllables, it is a single morpheme because it cannot be broken down into smaller meaningful parts.

Types of morphological processes

Inflection:

Systematic modification of a root form by means of prefixes and suffixes to indicate grammatical distinctions like singular and plural.

Stems: also called lemma, base form, root, lexeme

Doesn't change the word class

New grammatical role

Usually produces a predictable, non idiosyncratic change of meaning.

run → runs | running | ran

hope+ing → hoping hop → hopping

Derivation:

Ex: *compute* → *computer* → *computerization*

Less systematic than inflection

It can involve a change of meaning

Wide → *Widely*

Suffix *en* transforms adjective into verbs

Weak → *weaken*, *soft* → *soften*

Suffix *able* transforms verbs into adjective

Understand → *Understandable*

Suffix *er* transforms verbs into nouns (nominalization)

teach → *teacher*

Compounding:

Merging of two or more words into a new word

Downmarket, (to) overtake

Stemming

The removal of the inflectional ending from words (strip off any affixes)

Laughing, laugh, laughs, laughed → laugh

Problems

Can conflate semantically different words

Gallery and *gall* may both be stemmed to *gall*

A further step is to make sure that the resulting form is a known word in a dictionary, a task known as *lemmatization*.

Regular Expressions for Stemming

```
>>> re.findall(r'^(.)(ing|ly|ed|ious|ies|ive|es|s|ment)$', 'processing')  
[('process', 'ing')]
```

```
>>> re.findall(r'^(.*) (ing|ly|ed|ious|ies|ive|es|s|ment)$', 'processes')  
[('processe', 's')]
```

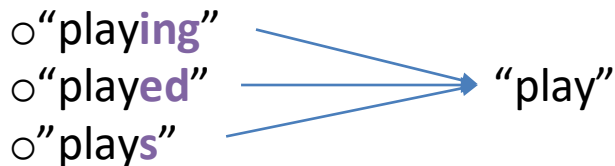
Note: the star operator is "greedy" and the `.*` part of the expression tries to consume as much of the input as possible. If we use the "non-greedy" version of the star operator, written `*?`, we get what we want:

```
>>> re.findall(r'^(.*)?(ing|ly|ed|ious|ies|ive|es|s|ment)$', 'processes')  
[('process', 'es')]
```

Stemming

Set of rules to slice a string to a substring that usually refers to a more general meaning.

- The goal is to remove word affixes (particularly suffixes) such as “s”, “es”, “ing”, “ed”, etc.



- The issue: It doesn't usually work with irregular forms such as irregular verbs: **“taught”**, **“brought”**, etc.

Stemming

Reduce terms to their stems in information retrieval

Stemming is crude chopping of affixes

language dependent

e.g., ***automate(s), automatic, automation*** all reduced to ***automat***.

*for example compressed
and compression are both
accepted as equivalent to
compress.*



for exampl compress and
compress ar both accept
as equival to compress

Stemming Algorithms

- Porter Stemmer
- Snowball Stemmer
- Lancaster Stemmer

Porter's algorithm

The most common English stemmer

Step 1a

sses → ss

ies → i

ss → ss

s → ∅

caresses → caress

ponies → poni

caress → caress

cats → cat

Step 1b

(*v*)ing → ∅ walking → walk

sing → sing

(*v*)ed → ∅ plastered → plaster

...

Step 2 (for long stems)

ational → ate

relational → relate

izer → ize

digitizer → digitize

ator → ate

operator → operate

...

Step 3 (for longer stems)

al → ∅ revival → reviv

able → ∅ adjustable → adjust

ate → ∅ activate → activ

...

Regular Expressions for Stemming

Let's define a function to perform stemming, and apply it to a whole text

```
>>> def stem(word):
...     regexp = r'^(.*?)(ing|ly|ed|ious|ies|ive|es|s|ment)?$'
...     stem, suffix = re.findall(regexp, word)[0]
...     return stem
...
>>> raw = """DENNIS: Listen, strange women lying in ponds distributing swords
... is no basis for a system of government. Supreme executive power derives from
... a mandate from the masses, not from some farcical aquatic ceremony."""
>>> tokens = nltk.word_tokenize(raw)
>>> [stem(t) for t in tokens]
['DENNIS', ':', 'Listen', ',', 'strange', 'women', 'ly', 'in', 'pond',
'distribut', 'sword', 'i', 'no', 'basi', 'for', 'a', 'system', 'of', 'govern',
',', 'Supreme', 'execut', 'power', 'deriv', 'from', 'a', 'mandate', 'from',
'the', 'mass', ',', 'not', 'from', 'some', 'farcical', 'aquatic', 'ceremony', '.']
```

- The RE removed the *s* from *ponds* but also from *is* and *basis*. It produced some non-words like *distribut* and *deriv*, but these are acceptable stems in some applications.

NLTK Stemmers

NLTK includes several off-the-shelf stemmers. The Porter and Lancaster stemmers follow their own rules for stripping affixes.

Stemming is not a well-defined process, and we typically pick the stemmer that best suits the application we have in mind.

```
>>> raw = """DENNIS: Listen, strange women lying in ponds distributing swords
... is no basis for a system of government. Supreme executive power derives from
... a mandate from the masses, not from some farcical aquatic ceremony."""
>>> tokens = nltk.word_tokenize(raw)
```

```
>>> porter = nltk.PorterStemmer()
>>> lancaster = nltk.LancasterStemmer()
>>> [porter.stem(t) for t in tokens]
['DENNI', ':', 'Listen', ',', 'strang', 'women', 'lie', 'in', 'pond',
'distribut', 'sword', 'is', 'no', 'basi', 'for', 'a', 'system', 'of', 'govern',
',', 'Suprem', 'execut', 'power', 'deriv', 'from', 'a', 'mandat', 'from',
'the', 'mass', ',', 'not', 'from', 'some', 'farcic', 'aquat', 'ceremoni', '.']
>>> [lancaster.stem(t) for t in tokens]
['den', ':', 'list', ',', 'strange', 'wom', 'lying', 'in', 'pond', 'distribut',
'sword', 'is', 'no', 'bas', 'for', 'a', 'system', 'of', 'govern', ',', 'suprem',
'execut', 'pow', 'der', 'from', 'a', 'mand', 'from', 'the', 'mass', ',', 'not',
'from', 'som', 'farc', 'aqu', 'ceremony', '.']
```

Porter Stemmer

Lexicon free stemmer

Rewrite rules

ATIONAL $\rightarrow$ ATE (e.g. *relational*, *relate*)

FUL $\rightarrow \epsilon$ (e.g. *hopeful*, *hope*)

SSES $\rightarrow$ SS (e.g. *caresses*, *caress*)

Errors of Commission

Organization $\rightarrow$ *organ*

Policy $\rightarrow$ *police*

Errors of Omission

Urgency (not stemmed to *urgent*)

European (not stemmed to *Europe*)

Is stemming useful?

For IR performance, some improvement (especially for smaller documents)

May help a lot for some queries, but on average (across all queries) it doesn't help much (i.e. for some queries the results are worse)

Word sense disambiguation on query terms: *business* may be stemmed to *busy*, *saw* (the tool) to *see*

Most studies for stemming for IR done for English (may help more for other languages)

The possibility of letting people interactively influence the stemming has not been studied much

Since improvement is small, often IR engine usually don't use stemming

Case folding

Applications like IR: reduce all letters to lower case

Since users tend to use lower case

Possible exception: upper case in mid-sentence?

e.g., ***General Motors***

Fed vs. ***fed***

SAIL vs. ***sail***

For sentiment analysis, Machine Translation ,
Information extraction

Case is helpful (***US*** versus ***us*** is important)

Lemmatization

Reduce inflections or variant forms to base form

am, are, is → be

car, cars, car's, cars' → car

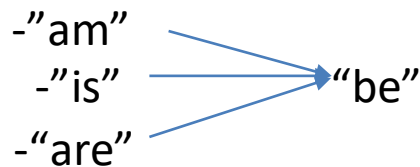
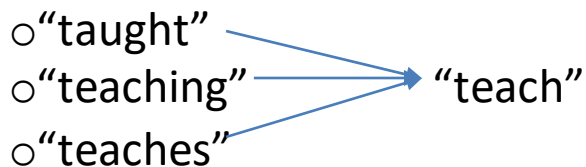
the boy's cars are different colors → the boy car be different color

Lemmatization: have to find correct dictionary headword form

Lemmatization

Similar to stemming, but **more advanced**. It uses a **look-up dictionary**.

- Handles more situations and usually works better than stemming.



- For the best results, correct word position tags should be provided: “adjective”, “noun”, “verb” etc.

Stemming vs. Lemmatization

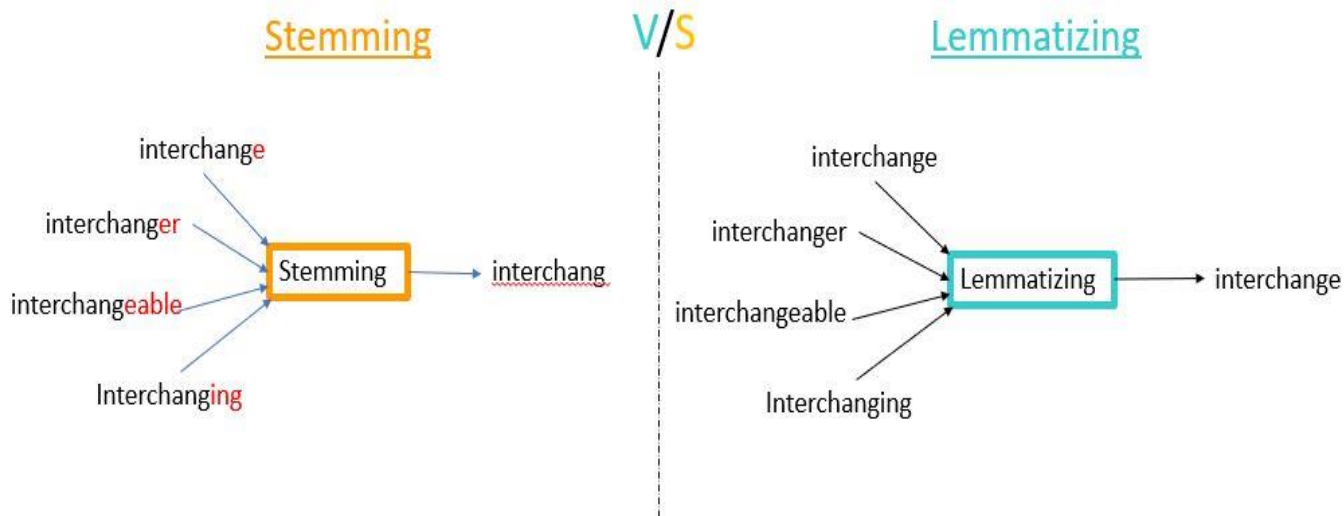
As we pointed out, lemmatization is a more complex method and usually works better.

E.g.,

Original sentence: "the children are playing outside. the weather was better yesterday."

▪ **Stemming** => "the children are **play** outside. the weather was better yesterday"

▪ **Lemmatization** => "the **child be play** outside. the weather **be good** yesterday"



Preprocessing: Stemming

Stemming & Lemmatization = Cut word down to base form

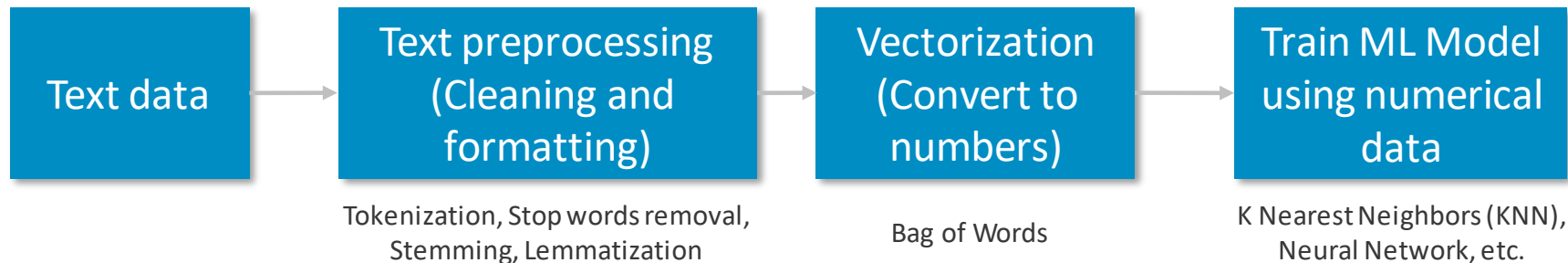
- Stemming: Uses rough heuristics to reduce words to base
- Lemmatization: Uses vocabulary and morphological analysis
- Makes the meaning of run, runs, running, ran all the same
- Cuts down on complexity by reducing the number of unique words

Multiple stemmers available in NLTK

- PorterStemmer, LancasterStemmer, SnowballStemmer
- WordNetLemmatizer

Machine Learning with Text Data

ML models need **well-defined numerical data**.



N-gram

- An n-gram is a sequence of n tokens from a given sample of text or speech.
- We can include n-grams in our term frequencies too.
- They are basically a set of co-occurring words within a given window and when computing the n-grams you typically move one word forward (although you can move X words forward in more advanced scenarios).

Sentence	1-gram (uni-gram):	2-gram (bi-gram):
It is not a dog, it is a wolf	"it", "is", "not", "a", "dog", "it", "is", "a", "wolf"	"it is", "is not", "not a", "a dog", "dog it", "it is", "is a", "a wolf"

For example, for the sentence “The cow jumps over the moon”.

If $N=3$, the n-grams would be:

1. the cow jumps
2. cow jumps over
3. jumps over the
4. over the moon

So you have 4 n-grams in this case. When $N=1$, this is referred to as **unigrams** and this is essentially the individual words in a sentence. When $N=2$, this is called **bigrams** and when $N=3$ this is called **trigrams**. When $N>3$ this is usually referred to as four grams or five grams and so on.

How many N-grams in a sentence?

If X =Num of words in a given sentence K , the number of n-grams for sentence K would be:

$$Ngrams_K = X - (N - 1)$$

- 1 - grams : Either my way or no way
- 2 - grams : Either my my way way or or no no way
- 3 - grams : Either my way my way or way or no or no way
- 4 - grams : Either my way or my way or no way or no way
- 5 - grams : Either my way or no my way or no way
- 6 - grams : Either my way or no way

Text	N-gram
Data	1-gram
Great information	2-gram
I am fine	3-gram
Nice to meet you	4-gram

Code: Tokenization (N-Grams)

Input:

```
from nltk.util import ngrams
```

```
my_words = word_tokenize(my_text) # This is the list of all words
```

```
twograms = list(ngrams(my_words,2)) # This is for two-word combos, but can pick any n
```

```
print(twograms)
```

Creating Vectors from Text

Here's a sample of reviews about a particular horror movie:

Review 1: This movie is very scary and long

Review 2: This movie is not scary and is slow

Review 3: This movie is spooky and good

Word Embedding is one such technique where we can represent the text using vectors. The more popular forms of word embeddings are:

BoW, which stands for Bag of Words

TF-IDF, which stands for Term Frequency-Inverse Document Frequency

Bag of Words (BoW)

- **Bag of Words** method converts text data into numbers.
- Bag of words is a Natural Language Processing technique of text modelling. In technical terms, we can say that it is a method of feature extraction with text data. This approach is a simple and flexible way of extracting features from documents.
- A bag of words is a representation of text that describes the occurrence of words within a document. We just keep track of word counts and disregard the grammatical details and the word order.
- It is called a “bag” of words because any information about the order or structure of words in the document is discarded. The model is only concerned with whether known words occur in the document, not where in the document.

Tokenization



Create
Vocabulary



Count Frequency
of Tokens



Create vectorized
representation

Why is the Bag-of-Words algorithm used?

- One of the biggest problems with text is that it is messy and unstructured, and machine learning algorithms prefer structured, well defined fixed-length inputs and by using the Bag-of-Words technique we can convert variable-length texts into a fixed-length vector.
- Also, at a much granular level, the machine learning models work with numerical data rather than textual data. So to be more specific, by using the bag-of-words (BoW) technique, we convert a text into its equivalent vector of numbers.

Bag of Words (BoW)

Simple example using word counts:

	a	cat	dog	is	it	my	not	old	wolf
	1	0	1	1	1	0	0	0	0
"It is a dog."	0	1	0	1	0	1	0	1	0
"my cat is old"	2	0	1	2	2	0	1	0	1

Bag of Words (BoW)

Simple example using word counts:

	a	cat	dog	is	it	my	not	old	wolf
"It is a dog."	1	0	1	1	1	0	0	0	0
"my cat is old"	0	1	0	1	0	1	0	1	0
"It is not a dog, it a is wolf."	2	0	1	2	2	0	1	0	1

Bag of Words (BoW)

Simple example using word counts:

	a	cat	dog	is	it	my	not	old	wolf
"It is a dog."	1	0	1	1	1	0	0	0	0
"my cat is old"	0	1	0	1	0	1	0	1	0
"It is not a dog, it a is wolf."	2	0	1	2	2	0	1	0	1

Bag of Words (BoW)

Simple example using word counts:

	a	cat	dog	is	it	my	not	old	wolf
"It is a dog."	1	0	1	1	1	0	0	0	0
"my cat is old"	0	1	0	1	0	1	0	1	0
"It is not a dog, it a is wolf."	2	0	1	2	2	0	1	0	1

Limitations of Bag-of-Words

it suffers from some shortcomings, such as:

1. **Vocabulary:** The vocabulary requires careful design, most specifically in order to manage the size, which impacts the sparsity of the document representations.
2. **Sparsity:** Sparse representations are harder to model both for computational reasons (space and time complexity) and also for information reasons, where the challenge is for the models to harness so little information in such a large representational space.
3. **Meaning:** Discarding word order ignores the context, and in turn meaning of words in the document (semantics). Context and meaning can offer a lot to the model, that if modeled could tell the difference between the same words differently arranged (“this is interesting” vs “is this interesting”), synonyms (“old bike” vs “used bike”), and much more.

Term Frequency-Inverse Document Frequency (TF-IDF)

Problem:

The scoring method being used above takes the count of each word and represents the word in the vector by the number of counts of that particular word. What does a word having high word count signify?

Does this mean that the word is important in retrieving information about documents?
The answer is NO.

If a word occurs many times in a document but also along with many other documents in our dataset, maybe it is because this word is just a frequent word; not because it is relevant or meaningful.

Solution:

One approach is to rescale the frequency of words by how often they appear in all documents so that the scores for frequent words like “the” that are also frequent across all documents are penalized.

This approach is called term frequency-inverse document frequency or shortly known as Tf-Idf approach of scoring.

TF-IDF is intended to reflect how relevant a term is in a given document.

Term Frequency: is a scoring of the frequency of the word in the current document.

Inverse Document Frequency: is a scoring of how rare the word is across documents.

Term Frequency-Inverse Document Frequency

$$\text{TF-IDF} = (\text{Term Frequency}) * (\text{Inverse Document Frequency})$$

Different value for every document / term combination

$$\frac{\text{Term Count in Document}}{\text{Total Terms in Document}} \log\left(\frac{\text{Total Documents}}{\text{Documents Containing the Term} + 1} \right) + 1$$

Also, note that the 1 added in the above formula is so that terms with zero IDF don't get suppressed entirely. This process is known as IDF smoothing.

Term Frequency-Inverse Document Frequency

Term Frequency

- So far, we've been recording the term (word) count

“This is an example”

This	is	an	example
1	1	1	1

- However, if there were two documents, one very long and one very short, it wouldn't be fair to compare them by word count alone
- A better way to compare them is by a normalized term frequency, which is (term count) / (total terms).

Term Frequency (TF)

Term frequency (TF): Increases the weight for **common** words in a document.

$$tf(term, doc) = \frac{\text{number of times the term occurs in the doc}}{\text{total number of terms in the doc}}$$

	a	cat	dog	is	it	my	not	old	wolf
"It is a dog."	0.25	0	0.25	0.25	0.25	0	0	0	0
"my cat is old"	0	0.25	0	0.25	0	0.25	0	0.25	0
"It is not a dog, it a is wolf."	0.22	0	0.11	0.22	0.22	0	0.11	0	0.11

Inverse Document Frequency (IDF)

term	idf
a	$\log(3/3)+1=1$
cat	$\log(3/2)+1=1.18$
dog	$\log(3/3)+1=1$
is	$\log(3/4)+1=0.87$
it	$\log(3/3)+1=1$
my	$\log(3/2)+1=1.18$
not	$\log(3/2)+1=1.18$
old	$\log(3/2)+1=1.18$
wolf	$\log(3/2)+1=1.18$

Inverse document frequency (IDF): Decreases the weights for **commonly** used words and **increases** weights for **rare** words in the vocabulary.

$$idf(term) = \log\left(\frac{n_{documents}}{n_{documents\ containing\ the\ term} + 1}\right) + 1$$

$$e.g. idf("cat") = 1.18$$

If the word is very common and appears in many documents, this number will approach 0. Otherwise, it will approach 1.

Term Frequency-Inverse Document Frequency

$$\text{TF-IDF} = (\text{Term Frequency}) * (\text{Inverse Document Frequency})$$

Different value for
every document /
term combination

Term Count in Document
Total Terms in Document

$\log\left(\frac{\text{Total Documents}}{\text{Documents Containing the Term} + 1}\right) + 1$

Term Frequency-Inverse Document Frequency

TF-IDF Intuition:

- TF-IDF assigns more weight to rare words and less weight to commonly occurring words.
- Tells us how frequent a word is in a document relative to its frequency in the entire corpus.
- Tells us that two documents are similar when they have more rare words in common.

Term Freq.-Inverse Doc. Freq (TF-IDF)

Term Freq. Inverse Doc. Freq (TF-IDF): Combines term frequency and inverse document frequency.

$$tf_{idf}(term, doc) = tf(term, doc) * idf(term)$$

	a	cat	dog	is	it	my	not	old	wolf
"It is a dog."	0.25	0	0.25	0.22	0.25	0	0	0	0
"my cat is old"	0	0.3	0	0.22	0	0.3	0	0.3	0
"It is not a dog, it a is wolf."	0.22	0	0.11	0.19	0.22	0	0.13	0	0.13

Count Vectorizer vs TF-IDF Vectorizer

```
import pandas as pd
corpus = ['This is the first document.',
          'This is the second document.',
          'And the third one. One is fun.']

# original Count Vectorizer
from sklearn.feature_extraction.text import CountVectorizer
cv = CountVectorizer()
X = cv.fit_transform(corpus).toarray()
pd.DataFrame(X, columns=cv.get_feature_names())

# new TF-IDF Vectorizer
from sklearn.feature_extraction.text import TfidfVectorizer
cv_tfidf = TfidfVectorizer()
X_tfidf = cv_tfidf.fit_transform(corpus).toarray()
pd.DataFrame(X_tfidf, columns=cv_tfidf.get_feature_names())
```

Count Vectorizer vs TF-IDF Vectorizer

Count Vectorizer Output:

	and	document	first	fun	is	one	second	the	third	this
0	0	1	1	0	1	0	0	1	0	1
1	0	1	0	0	1	0	1	1	0	1
2	1	0	0	1	1	2	0	1	1	0

TF-IDF Vectorizer Output:

	and	document	first	fun	is	one	second	the	third	this
0	0.00000	0.450145	0.591887	0.00000	0.349578	0.00000	0.000000	0.349578	0.00000	0.450145
1	0.00000	0.450145	0.000000	0.00000	0.349578	0.00000	0.591887	0.349578	0.00000	0.450145
2	0.36043	0.000000	0.000000	0.36043	0.212876	0.72086	0.000000	0.212876	0.36043	0.000000

Count Vectorizers: Count Vectorizer is a way to convert a given set of strings into a frequency representation.

Count Vectors can be helpful in understanding the type of text by the frequency of words in it.

But its major disadvantages are:

- I. Its inability in identifying more important and less important words for analysis.
- II. It will just consider words that are abundant in a corpus as the most statistically significant word.
- III. It also doesn't identify the relationships between words such as linguistic similarity between words.

TF-IDF is better than Count Vectorizers because it not only focuses on the frequency of words present in the corpus but also provides the importance of the words.

We can then remove the words that are less important for analysis, hence making the model building less complex by reducing the input dimensions.

TFIDF is based on the logic that words that are too abundant in a corpus and words that are too rare are both not statistically important for finding a pattern.

The Logarithmic factor in tfidf mathematically penalizes the words that are too abundant or too rare in the corpus by giving them low tfidf scores.

Higher value of tfidf signifies higher importance of the words in the corpus while lower values represent lower importance.

Dis-advantage is: It fails to provide linguistic information about the words such as the real meaning of the words, similarity with other words etc.

Minimum Edit Distance

Alignments and edit distance

optimal character alignment between two words (the one with the fewest character changes: the **minimum edit distance** or MED).

- Example: if all changes count equally, MED(**stall**, **table**) is 3:

S	T	A	L	L		
	T	A	L	L	deletion	
	T	A	B	L	substitution	
	T	A	B	L	E	insertion

Alignments and edit distance

optimal character alignment between two words (the one with the fewest character changes: the **minimum edit distance** or MED).

- Example: if all changes count equally, MED(stall, table) is 3:

S	T	A	L	L	
	T	A	L	L	deletion
	T	A	B	L	substitution
	T	A	B	L	E insertion

- Written as an alignment:

S	T	A	L	L	-
d	/	/	s	/	i
-	T	A	B	L	E

More alignments

- There may be multiple best alignments. In this case, two:

S	T	A	L	L	-
d	/	/	s	/	i
-	T	A	B	L	E

S	T	A	-	L	L
d	/	/	i	/	s
-	T	A	B	L	E

- And **lots** of non-optimal alignments, such as:

S	T	A	-	L	-	L
s	d	/	i	/	i	d
T	-	A	B	L	E	-

S	T	A	L	-	L	-
d	d	s	s	i	/	i
-	-	T	A	B	L	E

Edit Distance

- The minimum edit distance between two strings
- Is the minimum number of editing operations
 - Insertion
 - Deletion
 - Substitution
- Needed to transform one into the other

Minimum Edit Distance

- Two strings and their **alignment**:

I	N	T	E	*	N	T	I	O	N
*	E	X	E	C	U	T	I	O	N

Minimum Edit Distance

I N T E * N T I O N
| | | | | | | | | |
* E X E C U T I O N
d s s i s

- If each operation has cost of 1
 - Distance between these is 5
- If substitutions cost 2 (Levenshtein)
 - Distance between them is 8

How similar are two strings?

- Spell correction

- The user typed “graffe”

Which is closest?

- graf
 - graft
 - grail
 - giraffe

- Computational Biology

- Align two sequences of nucleotides

```
AGGCTATCACCTGACCTCCAGGCCGATGCCC  
TAGCTATCACGACCGCGGTTCGATTGCCCGAC
```

- Resulting alignment:

```
-AGGCTATCACCTGACCTCCAGGCCGA--TGCCC---  
TAG-CTATCAC--GACCGC--GGTCGATTGCCCGAC
```

- Also for Machine Translation, Information Extraction, Speech Recognition

How to find an optimal alignment

Brute force: Consider all possibilities, score each one, pick best. How many possibilities must we consider?

- First character could align to any of:

- - - - - T A B L E -

- Next character can align anywhere to its right
- And so on... the number of alignments grows exponentially with the length of the sequences.

Maybe not such a good method...

Defining Min Edit Distance

- For two strings
 - X of length n
 - Y of length m
- We define $D(i,j)$
 - the edit distance between $X[1..i]$ and $Y[1..j]$
 - i.e., the first i characters of X and the first j characters of Y
 - The edit distance between X and Y is thus $D(n,m)$

idea

Instead we will use a **dynamic programming** algorithm.

- Other DP (or **memoization**) algorithms: Viterbi, CKY.
- Used to solve problems where brute force ends up **recomputing** the same information many times.
- Instead, we
 - Compute the solution to each subproblem **once**,
 - Store (memoize) the solution, and
 - Build up solutions to larger computations by combining the pre-computed parts.

Dynamic Programming for Minimum Edit Distance

- **Dynamic programming:** A tabular computation of $D(n,m)$
- Solving problems by combining solutions to subproblems.
- Bottom-up
 - We compute $D(i,j)$ for small i,j
 - And compute larger $D(i,j)$ based on previously computed smaller values
 - i.e., compute $D(i,j)$ for all i ($0 < i < n$) and j ($0 < j < m$)

Defining Min Edit Distance (Levenshtein)

- Initialization

$$D(i, 0) = i$$

$$D(0, j) = j$$

- Recurrence Relation:

For each $i = 1 \dots M$

For each $j = 1 \dots N$

$$D(i, j) = \min \begin{cases} D(i-1, j) + 1 \\ D(i, j-1) + 1 \\ D(i-1, j-1) + \begin{cases} 2; & \text{if } X(i) \neq Y(j) \\ 0; & \text{if } X(i) = Y(j) \end{cases} \end{cases}$$

- Termination:

$D(N, M)$ is distance

The Edit Distance Table

N	9									
O	8									
I	7									
T	6									
N	5									
E	4									
T	3									
N	2									
I	1									
#	0	1	2	3	4	5	6	7	8	9
	#	E	X	E	C	U	T	I	O	N

The Edit Distance Table

N	9									
O	8									
I	7									
T	6									
N	5									
E	4									
T	3									
N	2									
I	1									
#	0	1	2	3	4	5	6	7	8	9
	#	E	X	E	C	U	T	I	O	N

$$D(i,j) = \min \begin{cases} D(i-1,j) + 1 \\ D(i,j-1) + 1 \\ D(i-1,j-1) + \begin{cases} 2; & \text{if } S_1(i) \neq S_2(j) \\ 0; & \text{if } S_1(i) = S_2(j) \end{cases} \end{cases}$$



Edit Distance

$$D(i,j) = \min \begin{cases} D(i-1,j) + 1 \\ D(i,j-1) + 1 \\ D(i-1,j-1) + \begin{cases} 2; & \text{if } S_1(i) \neq S_2(j) \\ 0; & \text{if } S_1(i) = S_2(j) \end{cases} \end{cases}$$

N	9									
O	8									
I	7									
T	6									
N	5									
E	4									
T	3									
N	2									
I	1									
#	0	1	2	3	4	5	6	7	8	9
	#	E	X	E	C	U	T	I	O	N

The Edit Distance Table

N	9	8	9	10	11	12	11	10	9	8
O	8	7	8	9	10	11	10	9	8	9
I	7	6	7	8	9	10	9	8	9	10
T	6	5	6	7	8	9	8	9	10	11
N	5	4	5	6	7	8	9	10	11	10
E	4	3	4	5	6	7	8	9	10	9
T	3	4	5	6	7	8	7	8	9	8
N	2	3	4	5	6	7	8	7	8	7
I	1	2	3	4	5	6	7	6	7	8
#	0	1	2	3	4	5	6	7	8	9
	#	E	X	E	C	U	T	I	O	N